

React Interview Prep

Topic 9 — React.memo: Complete Guide

Analogy · Problem · How It Works · 3 Scenarios · Custom Comparator · Trio · When NOT to Use · Q&A;

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 9

by Akshay Chavhan • React Interview Series

1. Like You're 5 — The Calculation Analogy

Mom asks 'What is 2+2?' → You say 4. She asks again → You don't recalculate. You remember the answer.

■ *React.memo does exactly this for components: Same props → don't re-render → reuse last result.*

2. The Problem React.memo Solves

By default, every time a parent re-renders, ALL child components re-render — even if their props didn't change.

```
function Child({ name }) {  
  console.log('Child rendered!'); // fires on EVERY parent re-render  
  return <p>Hello {name}</p>;  
}  
  
// Click Count button 10 times → 'Child rendered!' logs 10 times  
// Even though name='Akshay' never changed ■  
  
// ■ Fix with React.memo  
const Child = React.memo(function Child({ name }) {  
  console.log('Child rendered!'); // fires ONLY when name changes  
  return <p>Hello {name}</p>;  
});  
  
// Click Count 10 times → 'Child rendered!' logs 0 times ■
```

3. How React.memo Works — Shallow Comparison



Prop type	Compared by	Works with React.memo?
string, number, boolean	Value ===	■ Yes — always safe
Object { }	Reference ===	■ New ref every render = always changed
Array []	Reference ===	■ New ref every render = always changed
Function () => { }	Reference ===	■ New ref every render = always changed

■ *Shallow = checks each prop with ===. Primitives work fine. Objects/arrays/functions must be stabilized with useMemo / useCallback.*

4. The 3 Scenarios

Scenario 1 — Primitive props ■ Works perfectly

```
const Child = React.memo(({ name, count }) => <p>{name}: {count}</p>);  
// name='Akshay' count={5} — never change → Child NEVER re-renders ■
```

Scenario 2 — Object prop ■ Needs useMemo

■ Wrong

```
// New object every render  
// React.memo sees new ref  
// re-renders every time ■  
const user = { name: 'Akshay' };  
return <Child user={user} />;
```

■ Correct

```
// Stable reference with useMemo  
const user = useMemo(() => ({  
  name: 'Akshay'  
}), []);  
// React.memo sees same ref ■  
return <Child user={user} />;
```

Scenario 3 — Function prop ■ Needs useCallback

■ Wrong

```
// New function every render  
// React.memo re-renders ■  
const handleClick = () => {  
  console.log('clicked');  
};  
return <Child  
  onClick={handleClick} />;
```

■ Correct

```
// Stable reference with  
useCallback  
const handleClick = useCallback(  
  () => console.log('clicked'),  
  []  
);  
// React.memo works ■
```

■ **React.memo + useMemo + useCallback = the trio. memo checks props, useMemo stabilizes objects, useCallback stabilizes functions.**

5. Custom Comparison Function

Pass a second argument to React.memo to define exactly what props changed means.

```
const Child = React.memo(  
  ({ user }) => <p>{user.name}</p>,  
  (prevProps, nextProps) => {  
    // true → skip re-render (props 'equal')  
    // false → re-render (props 'different')  
    return prevProps.user.id === nextProps.user.id;  
    // only re-render if user ID changes — ignore other fields  
  }  
);
```

■■ **Be careful — if you forget to compare a prop, the component shows stale data.**

6. React.memo vs useMemo vs useCallback

Hook / API	Memoizes	Use for	Needs partner?
React.memo	A component	Skip child re-render if props same	useMemo + useCallback
useMemo	A value	Stable object/array ref, expensive calc	React.memo on child
useCallback	A function	Stable function ref passed as prop	React.memo on child

```
// They work together – all 3 needed for complete optimization:
function Parent() {
  const user = useMemo(() => ({ name: 'Akshay' }), []); // stable obj
  const handle = useCallback(() => console.log('hi'), []); // stable fn
  return <MemoChild user={user} onClick={handle} />; // all props stable ■
}
```

7. Proving It Works — React DevTools Profiler

Step	Action	What you see
1	Open React DevTools → Profiler tab	Flame graph view ready
2	Click Record	Recording starts
3	Click button that changes parent state	Parent re-renders
4	Stop recording	Flame graph shows
5 — Without memo	Parent coloured + Child coloured	Both re-rendered
5 — With memo (same props)	Parent coloured + Child grey	Child SKIPPED ■

■ Grey = memoized and skipped. Coloured = actually rendered. This is how you PROVE React.memo is working.

8. When NOT to Use React.memo

Situation	Use React.memo?	Why
Cheap component (e.g. span)	■ No	memo overhead costs MORE than the re-render
Props change on every render	■ No	memo never skips — pure wasted overhead
Parent never re-renders	■ No	No problem to solve in the first place
Component has heavy render output	■ Yes	Skipping expensive renders = real gain
Stable props + frequent parent updates	■ Yes	Child protected from parent noise

Can measure problem in Profiler	■ Yes	Always optimise based on evidence
---------------------------------	-------	-----------------------------------

■ *Measure first with React DevTools Profiler, then optimise. Never add `React.memo` blindly — it has its own memory and comparison cost.*

9. Interview Q&A;

Q

Beginner

Q1. What is React.memo?

→ React.memo is a Higher Order Component that wraps a component and memoizes its output. If the component receives the same props as the previous render, React skips re-rendering it and reuses the last output. It does a shallow comparison of props by default.

Q

Beginner

Q2. What is shallow comparison?

→ Shallow comparison checks each prop using strict equality `===`. Primitives like strings and numbers are compared by value — so they work correctly. Objects, arrays and functions are compared by reference — so even if their content is identical, a new reference means React.memo thinks the prop changed and re-renders.

Q

Intermediate

Q3. React.memo isn't working — what's likely wrong?

→ Almost always it's an unstable prop reference. Either an object or array created inline without `useMemo`, or a function created without `useCallback`. Every render creates a new reference — React.memo sees it as changed and re-renders. Fix: stabilize objects with `useMemo` and functions with `useCallback`.

Q

Intermediate

Q4. What is the difference between React.memo, useMemo and useCallback?

→ React.memo memoizes an entire component — skips re-rendering if props didn't change. `useMemo` memoizes a computed value inside a component. `useCallback` memoizes a function inside a component. They complement each other — `useMemo` and `useCallback` stabilize the props that React.memo then compares.

Q

Intermediate

Q5. When should you NOT use React.memo?

→ When the component is cheap to render — React.memo overhead can cost more than the re-render it prevents. Also avoid it when props always change every render — memo never skips and only adds cost. Only use when you can measure a real re-render performance problem in the Profiler.

Q

Expert

Q6. What is the custom comparison function in React.memo?

→ React.memo accepts a second argument — a function that receives previous and next props and returns true to skip re-render or false to re-render. It lets you define exactly what props changed means. Be careful — if you miss comparing a prop, the component can show stale UI data.

Q

Expert

Q7. How do you prove React.memo is working?

→ Use React DevTools Profiler. Record an interaction, check the flame graph. Without memo, both parent and child show coloured bars. With memo and same props, the child bar turns grey — meaning it was skipped. Grey = memoized and skipped. Coloured = actually rendered.